

Milestone 2 briefing — SD-specific PPM and OpenMP

This is the background for someone presenting slides 8–16 of `gPPM_Milestone2_Clean.pptx`. The slide-by-slide words to say are embedded as notes inside the PowerPoint (LibreOffice Impress: View → Notes). Read the Milestone 1 briefing first if the sequential reference is unfamiliar.

The assignment in one sentence

Starting from the unchanged Milestone 1 decoder, implement the paper's **Partitioned and Parallel Matrix (PPM)** method for SD codes: divide the recovery matrix into independently solvable pieces, change multiplication order for those pieces, run them concurrently with OpenMP, and compare work and elapsed time against the sequential reference.

This is **Milestone 2 PPM**, not the complete generalized gPPM algorithm. Here the choice of multiplication order is fixed. Dynamic cost-based choice, Reed–Solomon comparisons, and SIMD are planned for Milestone 3.

The problem PPM exploits

An SD stripe spans n disks and r sector rows. It tolerates m whole-disk failures plus s extra sector failures. The parity-check matrix H has $m \times r$ row-local equations and s dense equations that involve the entire stripe. This structure makes it different from a matrix with only global equations.

If a physical row has exactly m failed sectors, its own m local equations can solve those failures without results from another stripe row. That row becomes an **independent job** H_i . If a row has more than m failed sectors, its local equations are insufficient; its missing data goes into the **dependent remainder** H_{rest} , which also uses the dense parity equations.

Let z be the number of stripe rows containing the additional s sector failures in the deterministic layouts used for the experiments. Those rows are the dependent rows and the number of independent groups is $p = r - z$. Each independent group writes different faulty sectors and reads original survivors in its own row. The remainder must wait until the recovered independent sectors are available.

The small example that makes it concrete

The paper's Figure 3 uses $SD^{1,1}_{4,4}(8|1,2)$, with failed sectors $\{2, 6, 10, 13, 14\}$. Number a sector as $row \times 4 + disk$:

	disk 0	disk 1	disk 2	disk 3
row 0	0	1	[2]	3
row 1	4	5	[6]	7
row 2	8	9	[10]	11
row 3	12	[13]	[14]	15

Disk 2 is missing, plus sector 13 on disk 1. Here $m=1$, so rows 0, 1 and 2 each have one missing sector and one local equation. They form groups $\{2\}$, $\{6\}$, and $\{10\}$. The last row has two unknowns, $\{13, 14\}$, but only one local equation. Its remainder needs the dense equation as well. For instance, the first independent recovery is simply $B_2 = B_0 \text{ XOR } B_1 \text{ XOR } B_3$ for this coefficient choice.

The two changes to the computation

1. Matrix-first for independent jobs

Milestone 1 evaluates $BF = F^{-1} * S * BS$ in the normal order:

```
T = S*BS
BF = F^-1*T
counted region work = u(S) + u(F^-1)
```

For each independent Milestone 2 job, first multiply the **small coefficient matrices** instead:

```
G = F^-1*S
BF = G*BS
counted region work = u(G)
```

Associativity means the recovered data is identical. But field terms can cancel when forming G , leaving zero coefficients, which means fewer operations over large sectors. The paper's metric $u(M)$ is the count of nonzero entries that trigger the whole-sector `mult_XORs()` primitive; small coefficient-matrix multiplication is outside that metric.

This milestone always uses **matrix-first for independent groups**, and always uses the **normal sequence for Hrest**. A dynamic choice from $C = u + c * v$ is future generalized gPPM, not this implementation.

2. OpenMP for independent jobs

The order in `ppm_recover_sd()` is:

```
validate → partition → split and invert all jobs
           → run independent H_i in an OpenMP for loop
           → implicit barrier
           → solve Hrest sequentially
           → combine worker and remainder operation counts
```

Preparing matrices before any writes avoids invalidating inputs another job still needs to prepare. Each worker writes disjoint failed sectors, so no data lock is necessary. The barrier is essential: dense equations in the remainder read sectors the independent jobs have just reconstructed. The counter in `src/gf.c` uses C11 `_Thread_local` storage, avoiding a data race and avoiding an atomic update in every hot operation. OpenMP workers are capped at `min(requested_threads, p)`; three groups do not imply exactly three workers in every run.

What we implemented and where it lives

Part	Implementation
Small GF matrix product	<code>mat_mul()</code> in <code>src/matrix.c</code>
Matrix-first data decode	<code>ec_decode_matrix_first()</code> in <code>src/codec.c</code>
SD partition, job preparation, OpenMP, barrier, remainder	<code>ppm_recover_sd()</code> in <code>src/ppm.c</code>
Thread-local count	<code>gf_mult_xors_count</code> in <code>src/gf.c</code>
Figure 3 / field / thread tests	<code>tests/test_ppm.c</code>
Grid experiment	<code>src/ppm_sweep.c</code>
Timed baseline-versus-PPM experiment	<code>src/ppm_benchmark.c</code>

The sequential `ec_recover()` is still the Milestone 1 control. If someone asks “Where is the parallelism?”, point to the independent-job OpenMP `for` in `src/ppm.c`, not to the matrix multiplication utility itself.

Results to understand, not merely recite

The paper example: algorithmic work

On exactly the same failure pattern, the single coupled baseline requires $u(S)=22$ plus $u(F-1)=13$, so $C=35$. PPM partitions it into three independent jobs with three region calls each and a remainder with twenty:

```
baseline: 22 + 13      = 35 calls
PPM:      3 + 3 + 3 + 20 = 29 calls
difference: 6 calls, or 17.1% of the baseline work
```

This is an **operation count**, not “17.1% less elapsed time.” The gain comes from the algorithm even if only one OpenMP thread is requested.

Across the 7,833 evaluated (*configuration,z*) layouts, the median reduction in counted whole-region calls is 12.9%, with a range of 1.0–70.1%. Grouping by tolerated whole-disk failures gives medians 4.7% for $m=1$, 13.3% for $m=2$, and 22.1% for $m=3$. These cases use one representative deterministic placement per point on the full published coefficient grid; the count is about coverage of the work-reduction measurement, not a need to spend time defending correctness in the presentation.

The larger benchmark: elapsed time

The benchmark fixes $n=r=16$, $z=1$ and an exact 32 MiB codeword, tests all nine $m, s \in \{1, 2, 3\}$ settings, and compares the baseline with PPM at $T=1, 2, 4, 8$ requested OpenMP threads. It uses ten timed trials per setting, after warm-up. **Speedup is the median baseline time divided by the median PPM time in that same benchmark run.** This is not the mean of per-trial ratios, nor a comparison against the separate Milestone 1 benchmark run.

m	s	Field	PPM T=1	T=2	T=4	T=8
1	1	GF(2 ⁸)	1.13×	1.31×	1.34×	1.41×
1	2	GF(2 ¹⁶)	1.03×	1.10×	1.18×	1.22×
1	3	GF(2 ³²)	1.04×	1.14×	1.20×	1.18×
2	1	GF(2 ⁸)	1.02×	1.25×	1.65×	1.77×
2	2	GF(2 ¹⁶)	1.08×	1.35×	1.48×	1.46×
2	3	GF(2 ³²)	1.17×	1.36×	1.39×	1.49×
3	1	GF(2 ⁸)	1.19×	1.66×	2.10×	2.33×
3	2	GF(2 ¹⁶)	1.19×	1.53×	1.80×	1.67×
3	3	GF(2 ³²)	1.24×	1.53×	1.64×	1.73×

Slide 15 deliberately draws only $s=1$ for $m=1, 2, 3$ to give the audience three readable lines instead of nine overlapping line series. Its highest value is the $m=3, s=1, w=8, T=8$ median speedup of 2.33×. The $T=1$ result is important because it has no worker-to-worker concurrency; improvement there comes from partitioning and changed multiplication order. Higher T values add shared-memory parallelism.

Why isn't the eight-thread result eight times faster? The remainder is a sequential tail after the barrier. Starting/coordinating threads has cost, and multiple workers scan large sector buffers that compete for memory bandwidth. Some settings are best at four threads. Slide 16 draws a **schematic** of the dependency, not a measured parallel/serial fraction.

How to explain this milestone in 30 seconds

“The SD matrix has local parity rows and dense global rows. When a physical row has exactly m failures, its local equations solve that row on its own. We turn those rows into independent jobs, combine their small matrices before touching the sector data, and run them in OpenMP. The dependent rows wait at the barrier, then use the newly recovered sectors. In the paper's example, this changes 35 whole-sector operations into 29. Across our tested grid the median work reduction is 12.9%, and on the 32 MiB benchmark the best median speedup is 2.33× at eight requested threads.”

Questions a teammate may get

Independent of what? An H_i can be solved from the surviving sectors in its own physical stripe row and its local parity equations. It does not need another worker's recovered output.

Why is Hrest dependent? A row with extra failures has more unknowns than local equations. It needs dense equations involving the whole stripe, including independently recovered sectors after the jobs finish.

Why matrix-first only for the independent jobs? That is the fixed PPM strategy implemented for this milestone. The remainder runs normal decoding after the barrier; measuring both candidate costs and choosing automatically is part of planned gPPM.

Is T a matrix here? In performance results T is the requested OpenMP thread count. In Milestone 1's normal equation $T=S*BS$ is a temporary sector-region vector. The symbol is overloaded in two contexts.

Why don't the number of operations and speedup scale together exactly? Operation count excludes allocation, small matrix operations, scheduling, and bandwidth effects. Speedup times the full decoder call on real hardware.

Did we add SIMD or Reed-Solomon? No. Milestone 2 isolates SD partitioning, matrix-first for independent groups, and OpenMP. Milestone 3 extends the calculation choice and code family and adds vectorized arithmetic.

Inputs and reproduction

`data/FAST-Coefficients.txt` supplies published code coefficients. Deterministic pseudo-random bytes fill the sectors; deterministic failure placement makes comparisons repeatable. Useful commands:

```
make ppm-test      # Figure 3 and focused implementation checks
make ppm-sweep     # work reduction across feasible SD parameter layouts
make ppm-benchmark # timed baseline versus PPM, T=1,2,4,8
make plots         # regenerate presentation figures
```

Raw results are `results/ppm_sweep_results.csv` and `results/ppm_benchmark_results.csv`. For why $H*B=0$ holds and how F and S map to physical sectors, see the Milestone 1 PDF briefing or the project guide.